

California Housing Prediction Project

📌 Problem Statement (Abstract Example)

The rising demand for affordable housing in California has made accurate housing price prediction a critical task for policymakers, real estate businesses, and buyers. Traditional valuation methods rely heavily on manual appraisals, which can be subjective and inefficient. To address this, machine learning models can be applied to historical housing data to build predictive systems that estimate median house values more reliably.

In this project, we use the California Housing dataset to develop and evaluate regression models that predict housing prices based on features such as median income, average number of rooms, population, and geographic location. By doing so, we aim to demonstrate how data-driven approaches can enhance decision-making in the real estate sector, assist government planning, and provide insights into the factors that most influence housing prices.

```
In [62]: # import the libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import fetch_california_housing
%matplotlib inline
```

```
In [63]: # read the dataset
housing = fetch_california_housing()
print(housing)
```

```
{'data': array([[ 8.3252, 41., 6.98412698, ..., 2.55555556,
37.88, -122.23],
[ 8.3014, 21., 6.23813708, ..., 2.10984183,
37.86, -122.22],
[ 7.2574, 52., 8.28813559, ..., 2.80225989,
37.85, -122.24],
...,
[ 1.7, 17., 5.20554273, ..., 2.3256351,
39.43, -121.22],
[ 1.8672, 18., 5.32951289, ..., 2.12320917,
39.43, -121.32],
[ 2.3886, 16., 5.25471698, ..., 2.61698113,
39.37, -121.24]], shape=(20640, 8)), 'target': array([4.52
6, 3.585, 3.521, ..., 0.923, 0.847, 0.894], shape=(20640,)), 'frame': None, 'target_names': ['MedHouseVal'], 'feature_names': ['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup', 'Latitude', 'Longitude'], 'DESCR': '.. _california_housing_dataset:\n\nCalifornia Housing dataset\n-----\n\n**Data Set Characteristics:**\n\nNumber of Instances: 20640\n\nNumber of Attributes: 8 numeric, predictive attributes and the target\n\nAttribute Information:\n - MedInc median income in block group\n - HouseAge median house age in block group\n - AveRooms average number of rooms per household\n - AveBedrms average number of bedrooms per household\n - Population block group population\n - AveOccup average number of household members\n - Latitude block group latitude\n - Longitude block group longitude\n\nMissing Attribute Values: None\n\nThis dataset was obtained from the StatLib repository.\nhttps://www.dcc.fc.up.pt/~ltorgo/Regression/cal_housing.html\n\nThe target variable is the median house value for California districts, expressed in hundreds of thousands of dollars ($100,000).\n\nThis dataset was derived from the 1990 U.S. census, using one row per census block group. A block group is the smallest geographical unit for which the U.S. Census Bureau publishes sample data (a block group typically has a population of 600 to 3,000 people).\n\nA household is a group of people residing within a home. Since the average number of rooms and bedrooms in this dataset are provided per household, these columns may take surprisingly large values for block groups with few households and many empty houses, such as vacation resorts.\n\nIt can be downloaded/loaded using the\nfunc:`sklearn.datasets.fetch_california_housing` function.\n\n.. rubric:: References\n\n- Pace, R. Kelley and Ronald Barry, Sparse Spatial Autoregressions,\nStatistics and Probability Letters, 33:291-297, 1997.\n'}
```

```
In [4]: housing.keys()
```

```
Out[4]: dict_keys(['data', 'target', 'frame', 'target_names', 'feature_names', 'DESCR'])
```

```
In [5]: print(housing.DESCR)
```

```
.. _california_housing_dataset:
```

California Housing dataset

****Data Set Characteristics:****

:Number of Instances: 20640

:Number of Attributes: 8 numeric, predictive attributes and the target

:Attribute Information:

- MedInc median income in block group
- HouseAge median house age in block group
- AveRooms average number of rooms per household
- AveBedrms average number of bedrooms per household
- Population block group population
- AveOccup average number of household members
- Latitude block group latitude
- Longitude block group longitude

:Missing Attribute Values: None

This dataset was obtained from the StatLib repository.

https://www.dcc.fc.up.pt/~ltorgo/Regression/cal_housing.html

The target variable is the median house value for California districts, expressed in hundreds of thousands of dollars (\$100,000).

This dataset was derived from the 1990 U.S. census, using one row per census block group. A block group is the smallest geographical unit for which the U.S. Census Bureau publishes sample data (a block group typically has a population of 600 to 3,000 people).

A household is a group of people residing within a home. Since the average number of rooms and bedrooms in this dataset are provided per household, these columns may take surprisingly large values for block groups with few households and many empty houses, such as vacation resorts.

It can be downloaded/loaded using the

:func:`sklearn.datasets.fetch\_california\_housing` function.

.. rubric:: References

- Pace, R. Kelley and Ronald Barry, Sparse Spatial Autoregressions, Statistics and Probability Letters, 33:291-297, 1997.

```
In [6]: print(housing.data)
```

```

[[ 8.3252 41. 6.98412698 ... 2.55555556
 37.88 -122.23 ]
 [ 8.3014 21. 6.23813708 ... 2.10984183
 37.86 -122.22 ]
 [ 7.2574 52. 8.28813559 ... 2.80225989
 37.85 -122.24 ]
 ...
 [ 1.7 17. 5.20554273 ... 2.3256351
 39.43 -121.22 ]
 [ 1.8672 18. 5.32951289 ... 2.12320917
 39.43 -121.32 ]
 [ 2.3886 16. 5.25471698 ... 2.61698113
 39.37 -121.24 ]]

```

In [7]: `print(housing.target)`

```
[4.526 3.585 3.521 ... 0.923 0.847 0.894]
```

In [8]: `print(housing.feature_names)`

```
['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup', 'Latitude', 'Longitude']
```

Lets prepare the dataset

In [9]: `dataset = pd.DataFrame(housing.data, columns = housing.feature_names)`
`dataset.head()`

Out[9]:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.2
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.2
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.2
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.2
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.2

In [10]: `dataset['Price'] = housing.target`
`dataset.head()`

Out[10]:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.2
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.2
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.2
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.2
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.2

In [64]: `### First step of EDA`
`dataset.info()`

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 9 columns):
#   Column      Non-Null Count  Dtype
---  -
0   MedInc      20640 non-null  float64
1   HouseAge    20640 non-null  float64
2   AveRooms    20640 non-null  float64
3   AveBedrms   20640 non-null  float64
4   Population  20640 non-null  float64
5   AveOccup    20640 non-null  float64
6   Latitude    20640 non-null  float64
7   Longitude   20640 non-null  float64
8   Price       20640 non-null  float64
dtypes: float64(9)
memory usage: 1.4 MB

```

In [13]: `dataset.describe()`

Out[13]:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOc
count	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000	20640.000
mean	3.870671	28.639486	5.429000	1.096675	1425.476744	3.070
std	1.899822	12.585558	2.474173	0.473911	1132.462122	10.386
min	0.499900	1.000000	0.846154	0.333333	3.000000	0.692
25%	2.563400	18.000000	4.440716	1.006079	787.000000	2.429
50%	3.534800	29.000000	5.229129	1.048780	1166.000000	2.818
75%	4.743250	37.000000	6.052381	1.099526	1725.000000	3.282
max	15.000100	52.000000	141.909091	34.066667	35682.000000	1243.333

In [14]: `## Check the missing values`
`dataset.isnull().sum()`

Out[14]:

```

MedInc      0
HouseAge    0
AveRooms    0
AveBedrms   0
Population  0
AveOccup    0
Latitude    0
Longitude   0
Price       0
dtype: int64

```

In [15]: `## EDA`
`## Check the Relationship`
`dataset.corr()`

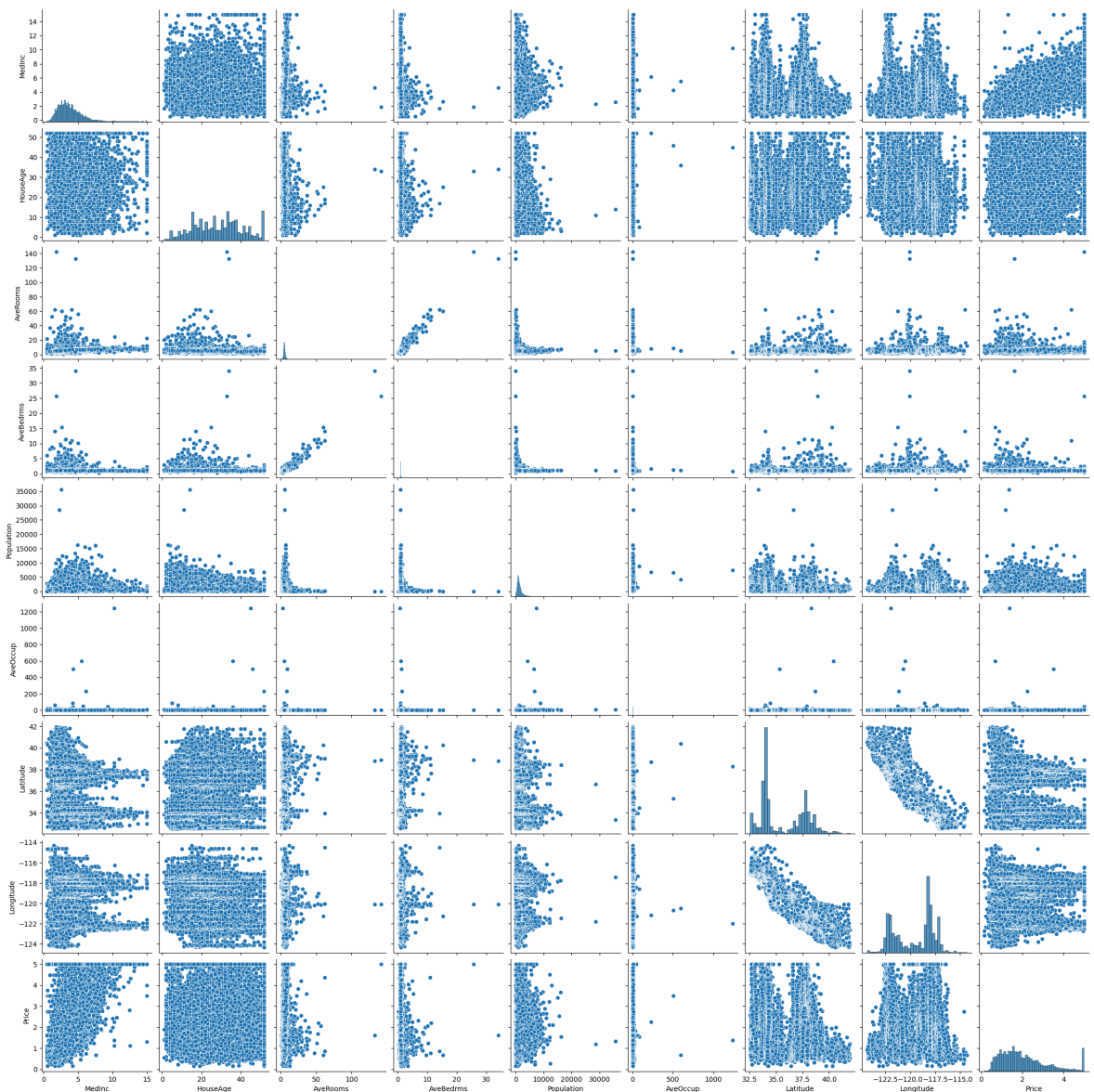
Out[15]:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude
MedInc	1.000000	-0.119034	0.326895	-0.062040	0.004834	0.018766	-0.0798
HouseAge	-0.119034	1.000000	-0.153277	-0.077747	-0.296244	0.013191	0.0111
AveRooms	0.326895	-0.153277	1.000000	0.847621	-0.072213	-0.004852	0.1063
AveBedrms	-0.062040	-0.077747	0.847621	1.000000	-0.066197	-0.006181	0.0697
Population	0.004834	-0.296244	-0.072213	-0.066197	1.000000	0.069863	-0.1087
AveOccup	0.018766	0.013191	-0.004852	-0.006181	0.069863	1.000000	0.0023
Latitude	-0.079809	0.011173	0.106389	0.069721	-0.108785	0.002366	1.0000
Longitude	-0.015176	-0.108197	-0.027540	0.013344	0.099773	0.002476	-0.9246
Price	0.688075	0.105623	0.151948	-0.046701	-0.024650	-0.023737	-0.1441



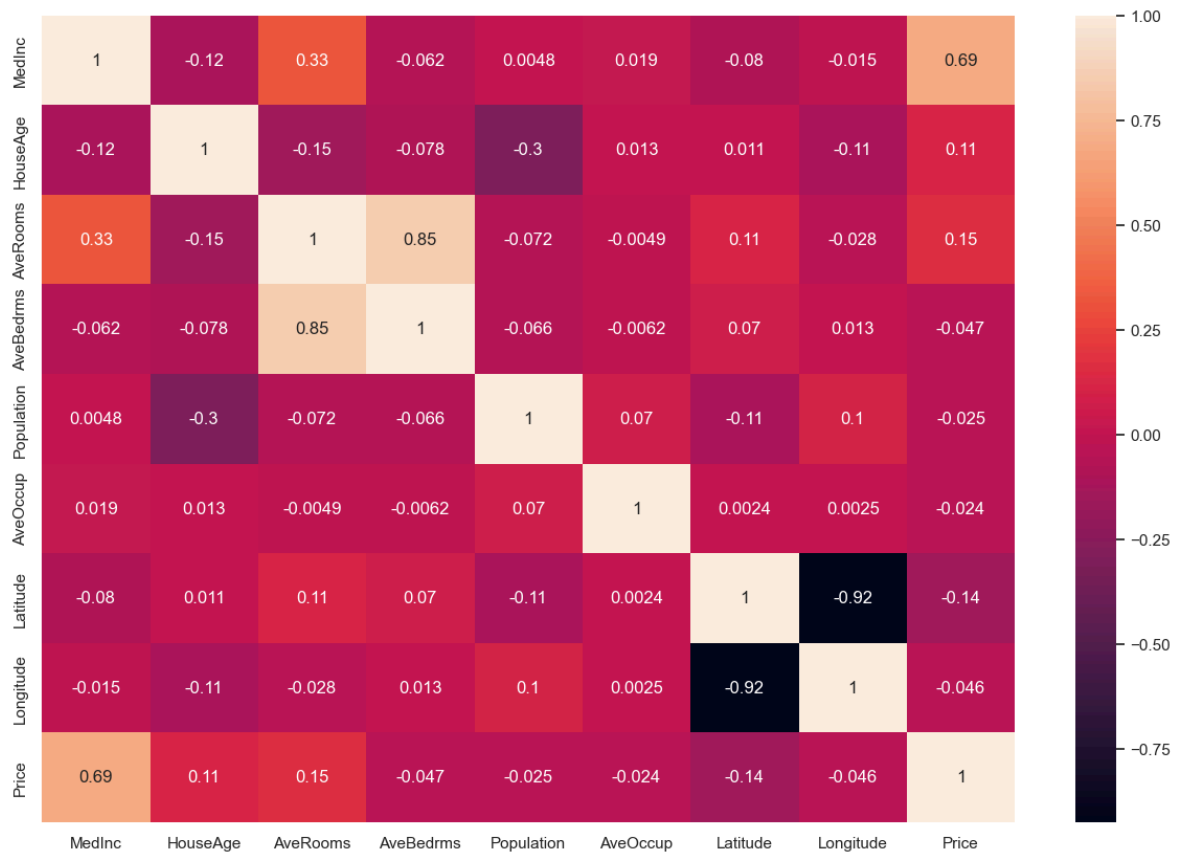
In [16]: `sns.pairplot(dataset)`

Out[16]: `<seaborn.axisgrid.PairGrid at 0x1c5066ab820>`



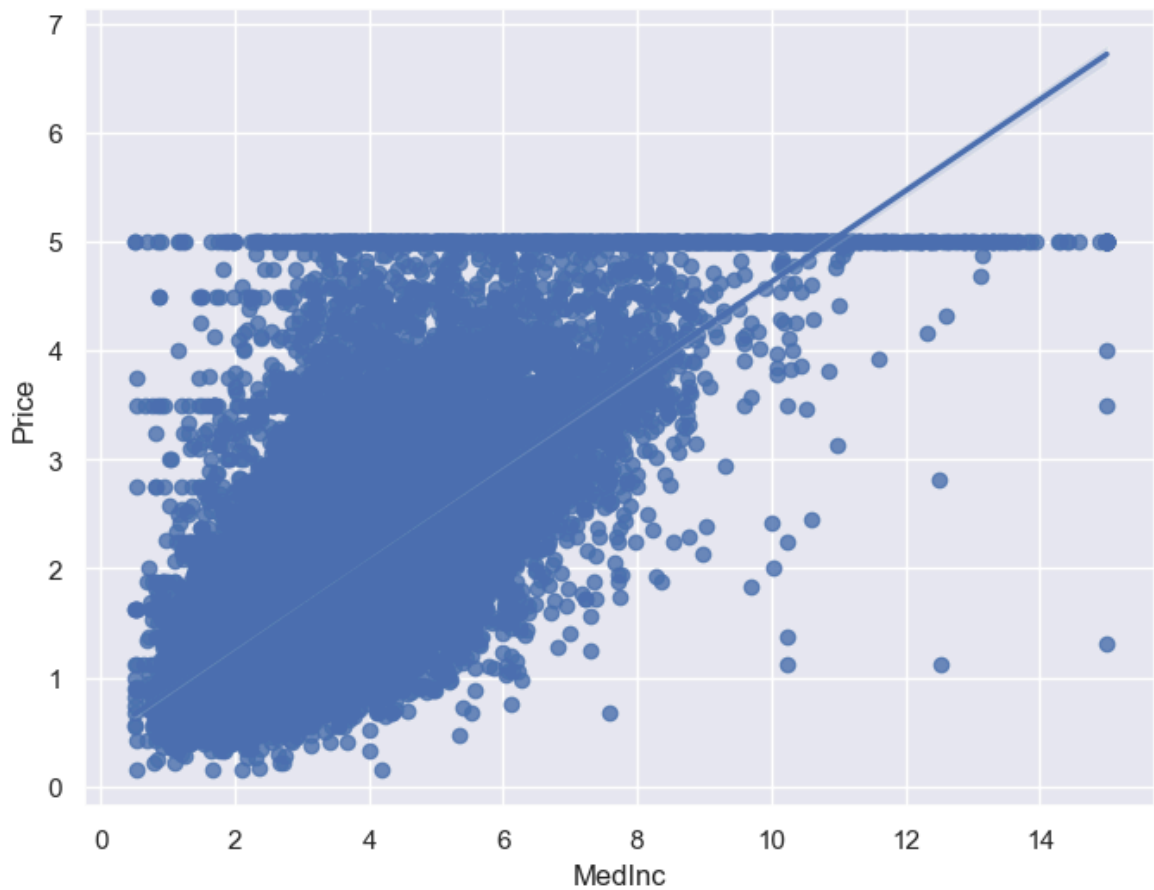
```
In [17]: sns.set(rc={'figure.figsize':(15,10)})
sns.heatmap(dataset.corr(), annot = True)
```

Out[17]: <Axes: >



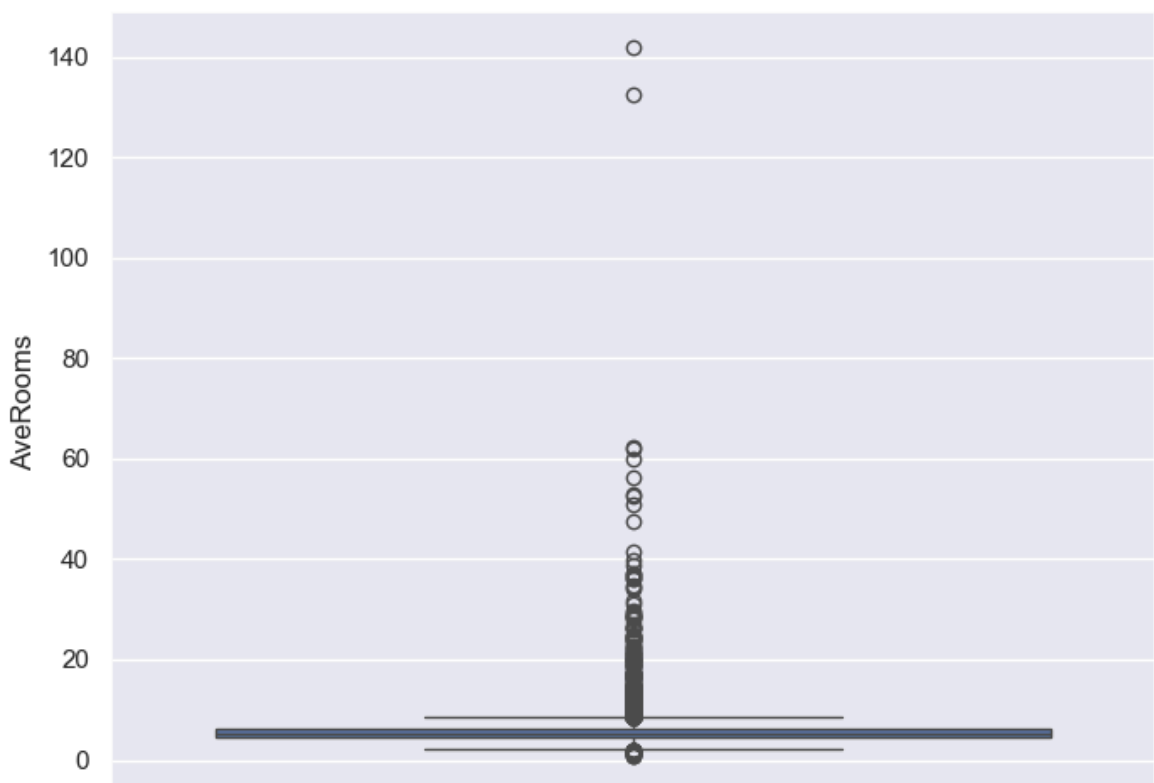
```
In [65]: sns.regplot(x = 'MedInc', y = 'Price', data = dataset)
```

Out[65]: <Axes: xlabel='MedInc', ylabel='Price'>



```
In [20]: sns.boxplot(dataset['AveRooms'])
```

```
Out[20]: <Axes: ylabel='AveRooms'>
```



```
In [21]: ## Independent And Dependet Features
X = dataset.iloc[:, :-1]
y = dataset.iloc[:, -1]
```



```
In [22]: X.head()
```

```
Out[22]:
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.2
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.2
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.2
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	-122.2
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	-122.2

◀ ————— ▶

```
In [23]: y
```

```
Out[23]:
```

0	4.526
1	3.585
2	3.521
3	3.413
4	3.422
	...
20635	0.781
20636	0.771
20637	0.923
20638	0.847
20639	0.894

Name: Price, Length: 20640, dtype: float64

Split the data Test and Train

```
In [24]: from sklearn.model_selection import train_test_split
```

```
In [25]: X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.33, random_state=42)
```

```
In [26]: print(X_train.shape, X_test.shape)
```

(13828, 8) (6812, 8)

```
In [27]: ## Standardize the dataset or feature scaling the data dataset
        from sklearn.preprocessing import StandardScaler
```

```
In [28]: scaler = StandardScaler()
```

```
In [29]: scaler
```

```
Out[29]:
```

▼ StandardScaler ⓘ ?

► Parameters

```
In [30]: X_train = scaler.fit_transform(X_train)
```

```
In [31]: X_test = scaler.fit_transform(X_test)
```

```
In [32]: X_train
```

```
Out[32]: array([[ -1.52169954, -0.75868455, -0.91813082, ..., -0.04014979,
                -0.78315293,  0.65090402],
                [ 0.18205382,  0.3523991 ,  0.30748451, ..., -0.05943619,
                0.8475975 , -1.31911678],
                [-0.17232772,  1.06666717, -0.25107784, ...,  0.09515433,
                -0.87218816,  0.66087881],
                ...,
                [-0.49518065,  0.59048846, -0.59192622, ...,  0.01608523,
                -0.75972261,  0.60103008],
                [ 0.96816708, -1.07613702,  0.39144665, ...,  0.0039722 ,
                0.90383028, -1.18445713],
                [-0.68377427,  1.86029835, -0.83041358, ..., -0.08065717,
                0.9928655 , -1.41387727]], shape=(13828, 8))
```

```
In [33]: X_test
```

```
Out[33]: array([[ -1.15004883, -0.30283646, -0.48402456, ...,  0.62488783,
                0.21852245,  0.26423782],
                [-0.70099345,  0.09547254, -0.14831375, ..., -0.18459773,
                -0.2115367 ,  0.03922932],
                [-0.19980096,  1.84803215, -0.56924017, ..., -1.07642092,
                1.03189519, -1.45082702],
                ...,
                [-0.01206507, -1.09945447,  0.39868138, ...,  0.43885168,
                1.45727978, -0.93080736],
                [ 0.28390685, -1.49776347,  0.29357284, ...,  0.56268512,
                -0.70236507,  1.07426845],
                [-0.58911258,  1.76837035, -0.61679693, ..., -0.45095617,
                1.02254608, -1.46582758]], shape=(6812, 8))
```

Model Training

1. Linear Regression

```
In [34]: from sklearn.linear_model import LinearRegression
         regression = LinearRegression()
```

```
In [35]: regression
```

```
Out[35]: ▼ LinearRegression ⓘ ?
         ► Parameters
```

```
In [36]: regression.fit(X_train, y_train)
```

```
Out[36]: ▼ LinearRegression ⓘ ?
         ► Parameters
```

```
In [37]: ## print the coefficient and the intercept
         print(regression.coef_)
```

```
[ 8.46603472e-01  1.20333548e-01 -2.98800785e-01  3.47243173e-01  
 -8.81413334e-05 -4.17242067e-02 -8.94420371e-01 -8.70401317e-01]
```

```
In [38]: print(regression.intercept_)
```

```
2.0666362048018536
```

```
In [39]: ## Prediction for the test data  
reg_pred = regression.predict(X_test)  
reg_pred
```

```
Out[39]: array([0.70176182, 1.75672011, 2.73470125, ..., 1.25697059, 1.65515384,  
                2.28422378], shape=(6812,))
```

Assumption of Linear Regression (Evaluating whether the linear model is good or not.)

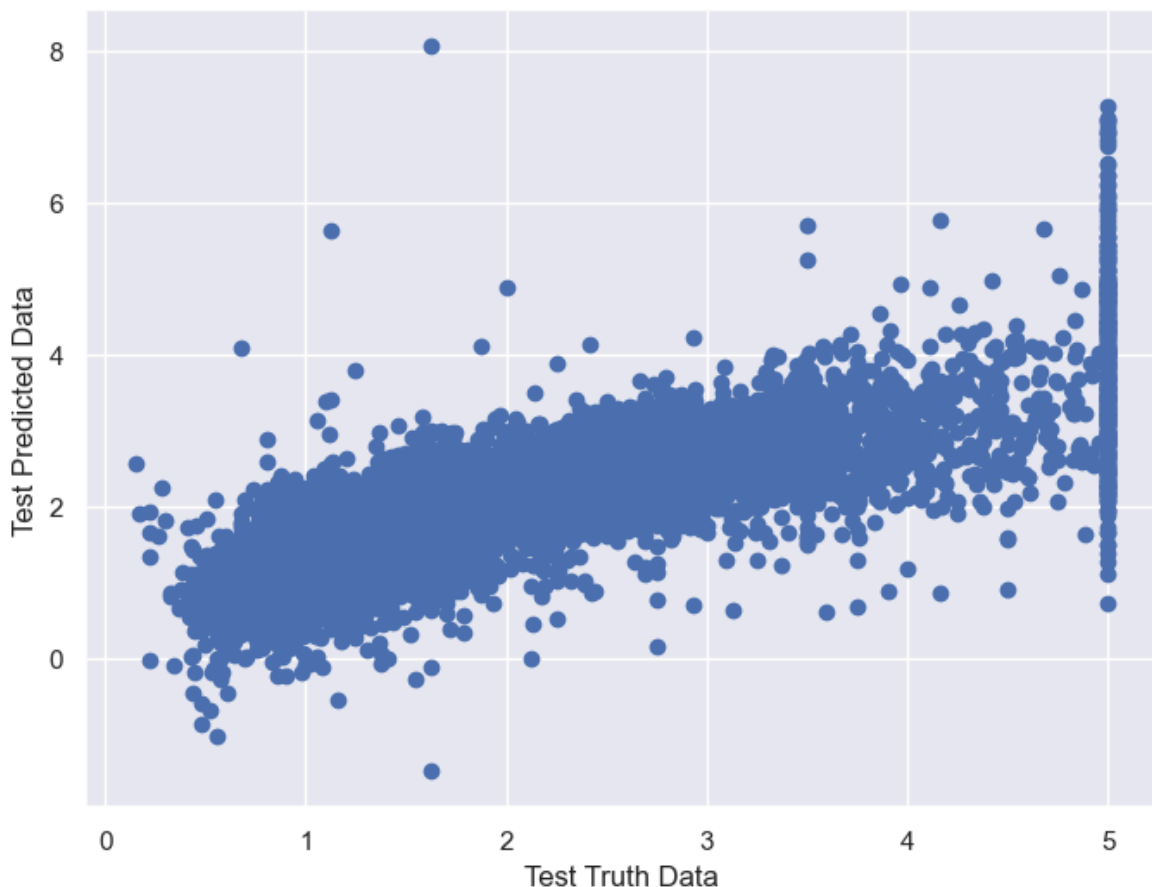
1. Plot y_{test} vs. predicted values in a scatter plot – it should show a strong linear relationship.

2. Plot the distribution of residual errors – it should approximately follow a normal distribution.

3. The scatter plot of predicted values and residuals shows a uniform distribution.

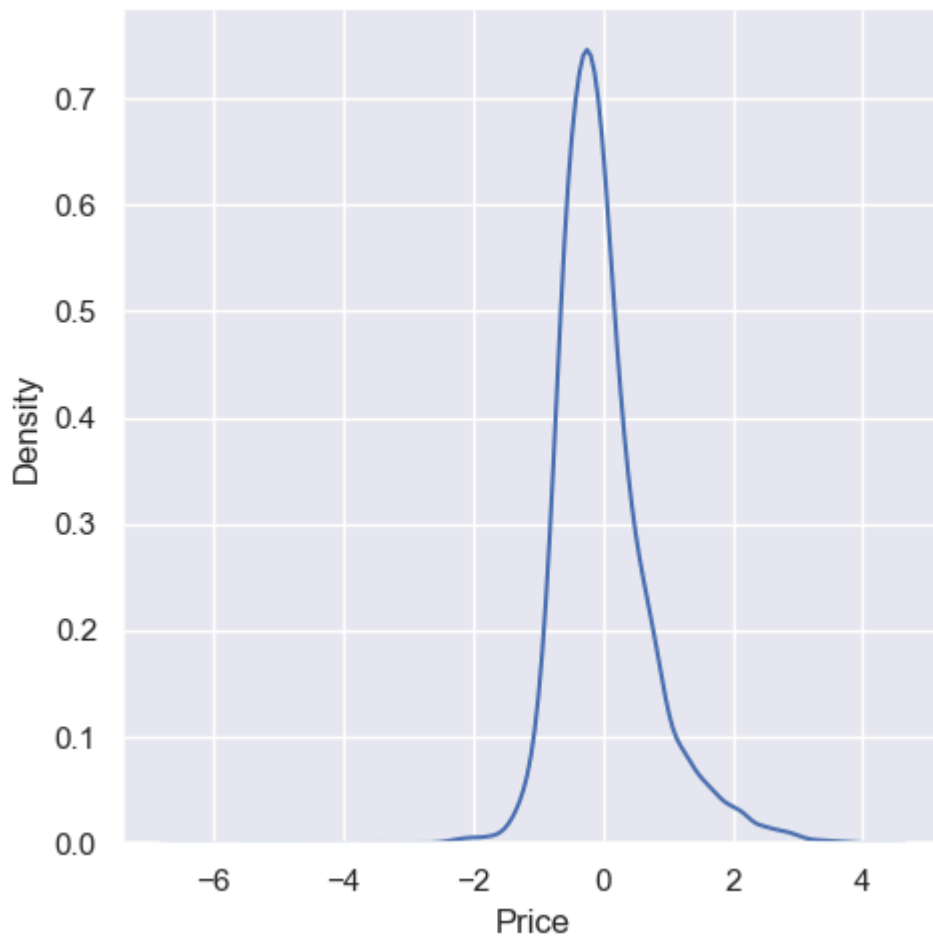
```
In [41]: ## 1. Plot y_test vs. predicted values in a scatter plot - it should show a strong  
plt.scatter(y_test, reg_pred)  
plt.xlabel("Test Truth Data")  
plt.ylabel("Test Predicted Data")
```

```
Out[41]: Text(0, 0.5, 'Test Predicted Data')
```

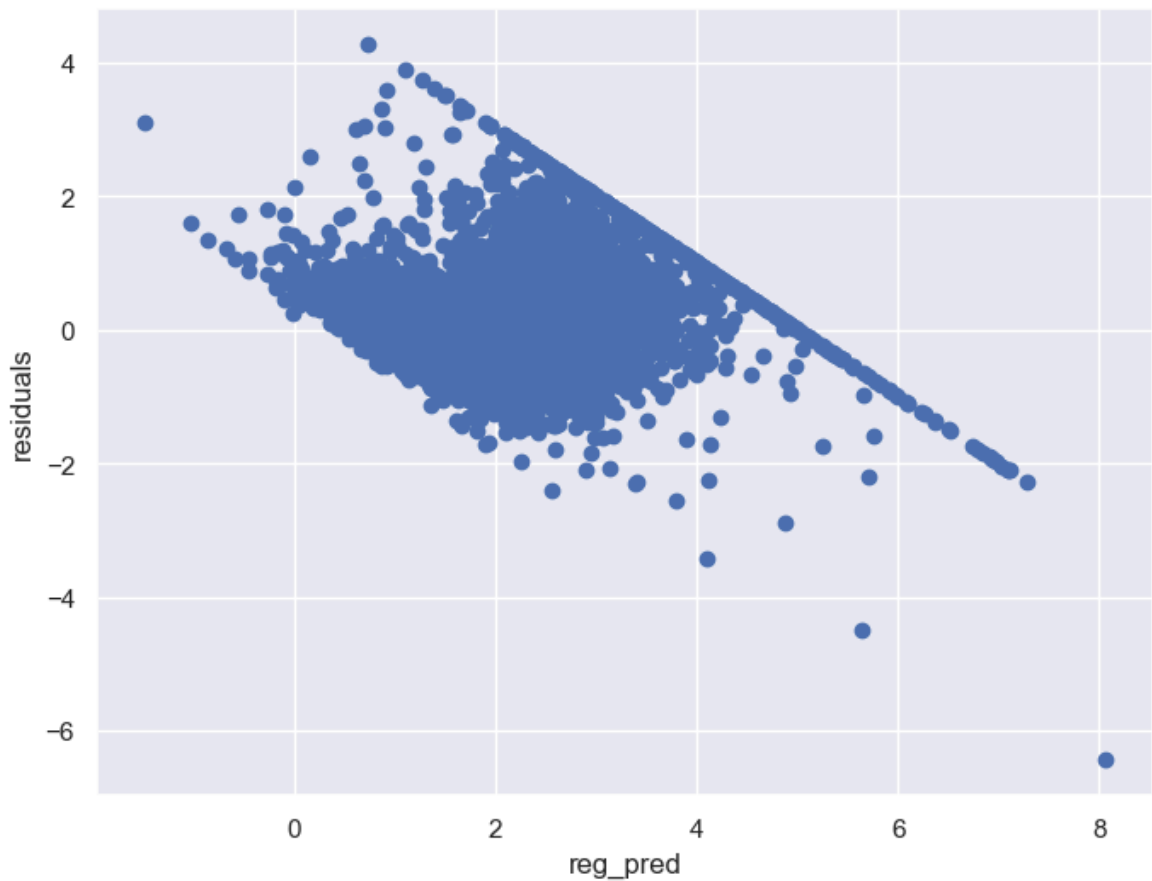


```
In [42]: ## residuals  
## 2. Plot the distribution of residual errors - it should approximately follow  
residuals = y_test - reg_pred  
sns.displot(residuals, kind="kde")
```

Out[42]: <seaborn.axisgrid.FacetGrid at 0x1c5891cfaf0>



```
In [43]: ## 3. Scattter plot with prediction and residual  
## uniform distribution  
plt.scatter(reg_pred, residuals)  
plt.xlabel("reg_pred")  
plt.ylabel("residuals")  
plt.show()  
#No, this is not a uniform distribution.  
#It looks more like a concentrated triangular distribution along a sloping line  
# no shape randomly distribution called uniform distribution
```



Performance Metrics

```
In [44]: from sklearn.metrics import mean_squared_error
from sklearn.metrics import mean_absolute_error
print(mean_squared_error(y_test, reg_pred))
print(mean_absolute_error(y_test, reg_pred))
print(np.sqrt(mean_squared_error(y_test, reg_pred)))
```

```
0.5231851999428957
0.525675206895467
0.7233154221658042
```

```
In [45]: ## R Square and adjusted R square
from sklearn.metrics import r2_score
score = r2_score(y_test, reg_pred)
print(score)
```

```
0.6073927560063519
```

```
In [46]: ## Adjusted R Squared
1 - (1 - score) * (len(y_test) - 1) / (len(y_test) - X_test.shape[1] - 1)
```

```
Out[46]: 0.6069310688165901
```

```
In [47]: # task
# 1. apply lasso and ridge algorithms also same data set elasticnet
```

2. Ridge Regression

```
In [48]: ## Ridge
from sklearn.linear_model import Ridge
ridge = Ridge()
ridge.fit(X_train, y_train)
```

```
Out[48]: ▼ Ridge ⓘ ⓘ
          ► Parameters
```

```
In [49]: ridge.coef_
```

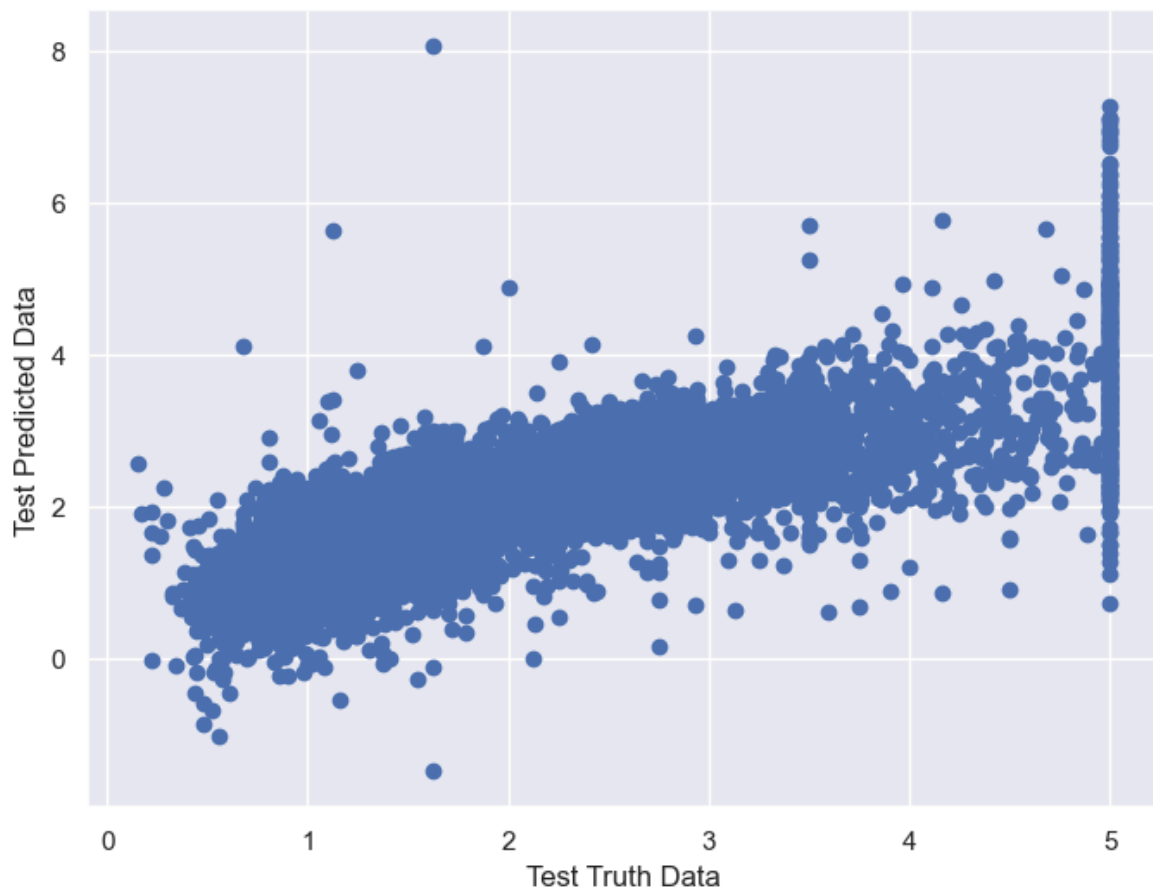
```
Out[49]: array([ 8.46533318e-01,  1.20427486e-01, -2.98545172e-01,  3.46925883e-01,
                -5.66543898e-05, -4.17290613e-02, -8.93520396e-01, -8.69488758e-01])
```

```
In [50]: ridge_pred = ridge.predict(X_test)
ridge_pred
```

```
Out[50]: array([0.70216823, 1.75653157, 2.73429552, ..., 1.25749782, 1.65552284,
                2.28389876], shape=(6812,))
```

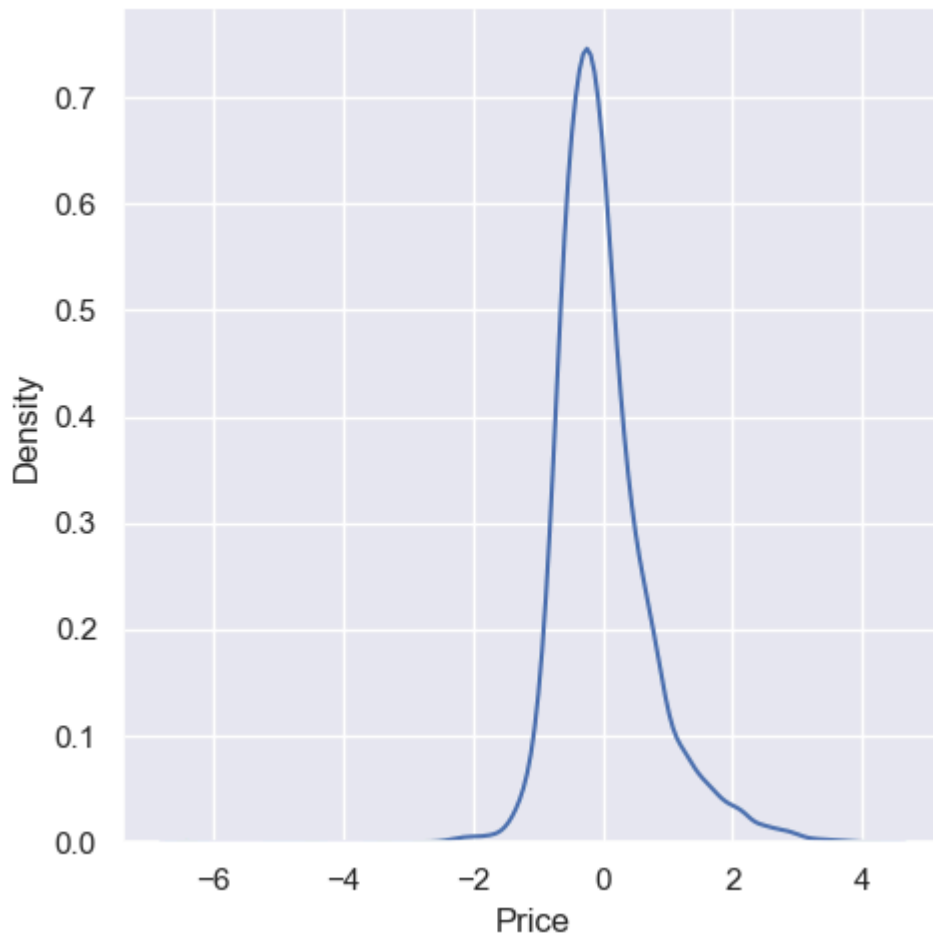
```
In [51]: plt.scatter(y_test, ridge_pred)
plt.xlabel("Test Truth Data")
plt.ylabel("Test Predicted Data")
```

```
Out[51]: Text(0, 0.5, 'Test Predicted Data')
```

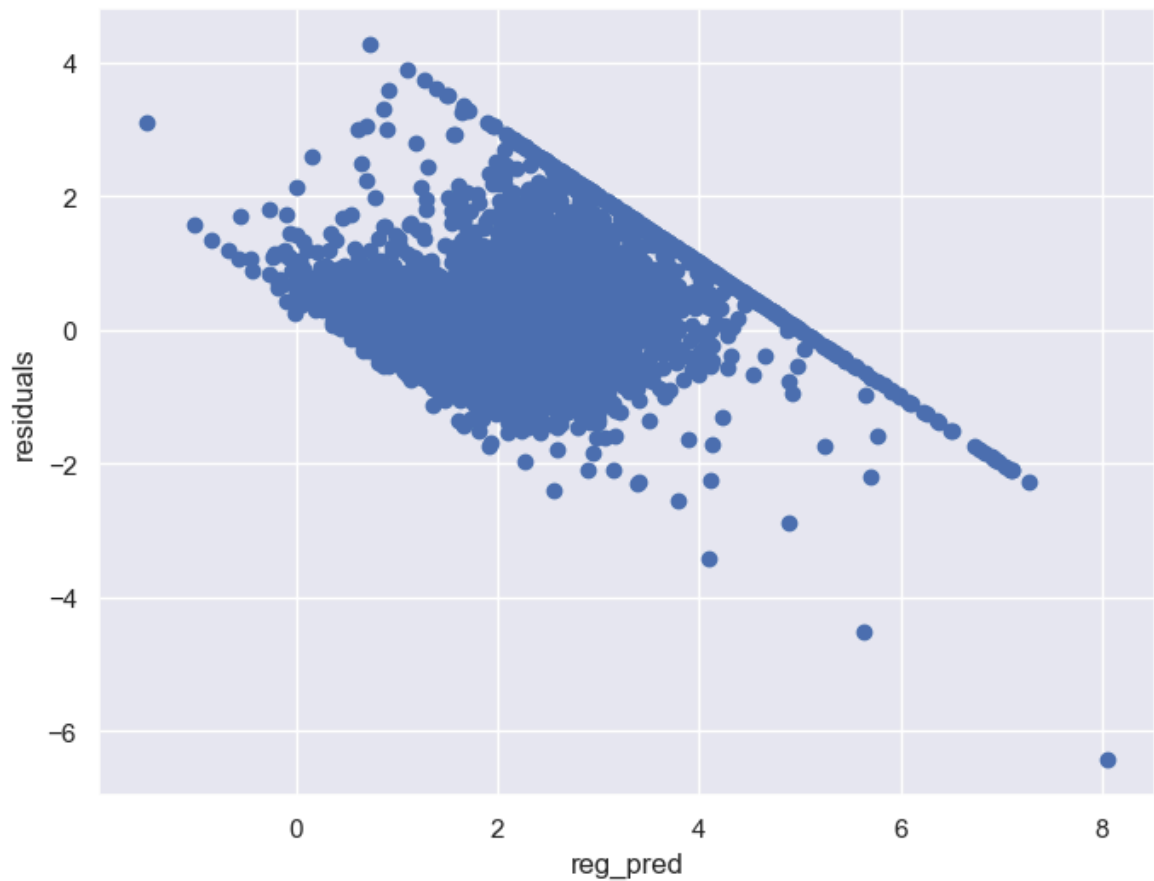


```
In [52]: ## residuals
## 2. Plot the distribution of residual errors - it should approximately follow
residuals = y_test - ridge_pred
sns.displot(residuals, kind="kde")
```

Out[52]: <seaborn.axisgrid.FacetGrid at 0x1c5892916c0>



```
In [53]: ## 3. Scattter plot with prediction and residual  
## uniform distribution  
plt.scatter(ridge_pred, residuals)  
plt.xlabel("reg_pred")  
plt.ylabel("residuals")  
plt.show()  
#No, this is not a uniform distribution.  
#It looks more like a concentrated triangular distribution along a sloping line  
# no shape randomly distribution called uniform distribution
```



```
In [54]: print(mean_squared_error(y_test, ridge_pred))
print(mean_absolute_error(y_test, ridge_pred))
print(np.sqrt(mean_squared_error(y_test, ridge_pred)))
```

```
0.5231735520208265
0.525668706826915
0.7233073703625772
```

```
In [55]: ## R Square and adjusted R square
from sklearn.metrics import r2_score
score = r2_score(y_test, ridge_pred)
print(score)
```

```
0.6074014968090016
```

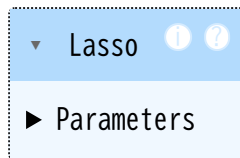
3. Lasso

```
In [71]: from sklearn.linear_model import Lasso
lasso = Lasso(alpha = 0.01)
lasso
```

```
Out[71]: ▾ Lasso ⓘ ⓘ
          ► Parameters
```

```
In [72]: lasso.fit(X_train, y_train)
```


Out[72]:



In [73]: `lasso.intercept_`

Out[73]: `np.float64(2.0666362048018536)`

In [74]: `lasso.coef_`

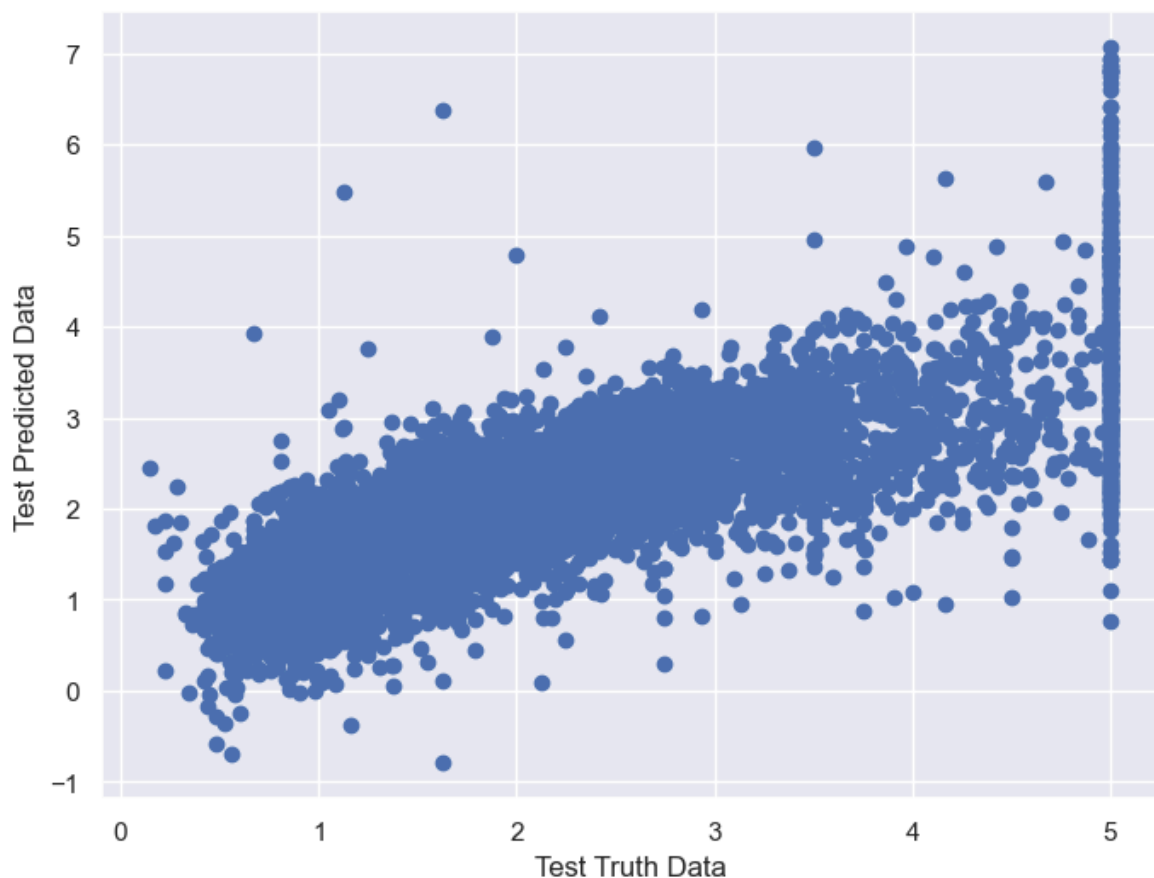
Out[74]: `array([0.79296204, 0.12425208, -0.16249723, 0.20902305, 0. ,
 -0.03116597, -0.78933429, -0.75764998])`

In [75]: `lasso_pred = lasso.predict(X_test)`
`lasso_pred`

Out[75]: `array([0.77471429, 1.72886619, 2.58468052, ..., 1.37504915, 1.74875524,
 2.20029786], shape=(6812,))`

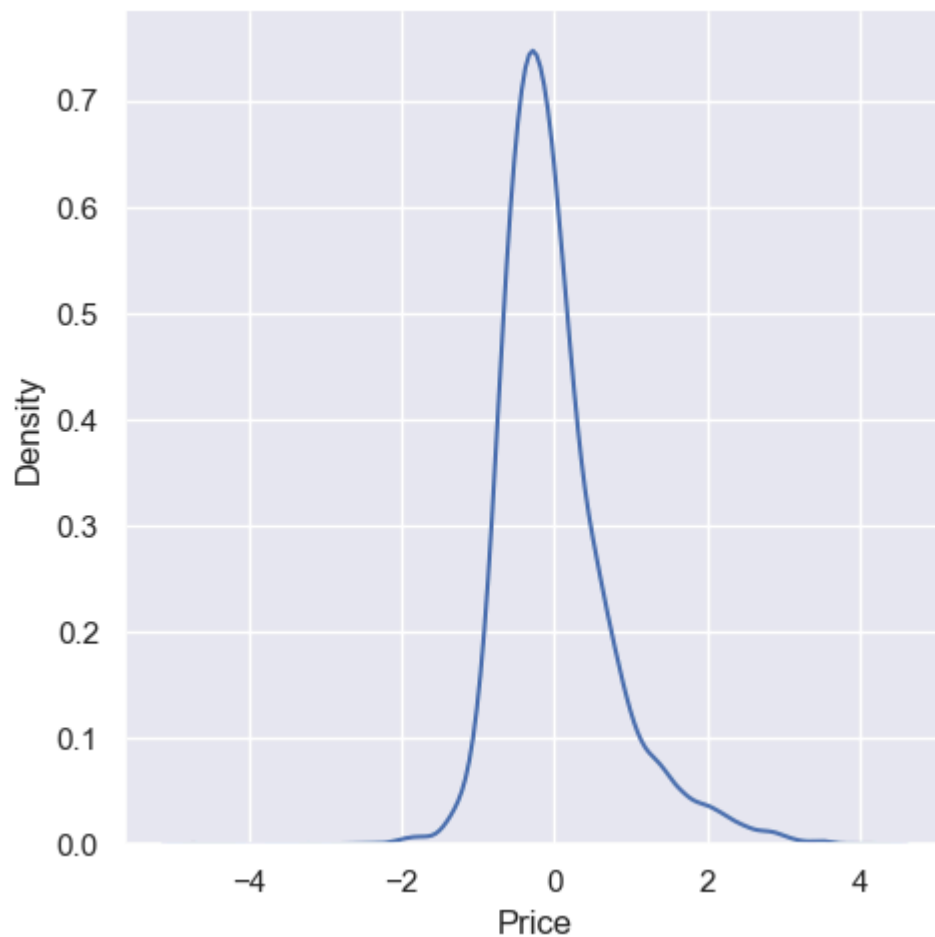
In [76]: `plt.scatter(y_test, lasso_pred)`
`plt.xlabel("Test Truth Data")`
`plt.ylabel("Test Predicted Data")`

Out[76]: `Text(0, 0.5, 'Test Predicted Data')`

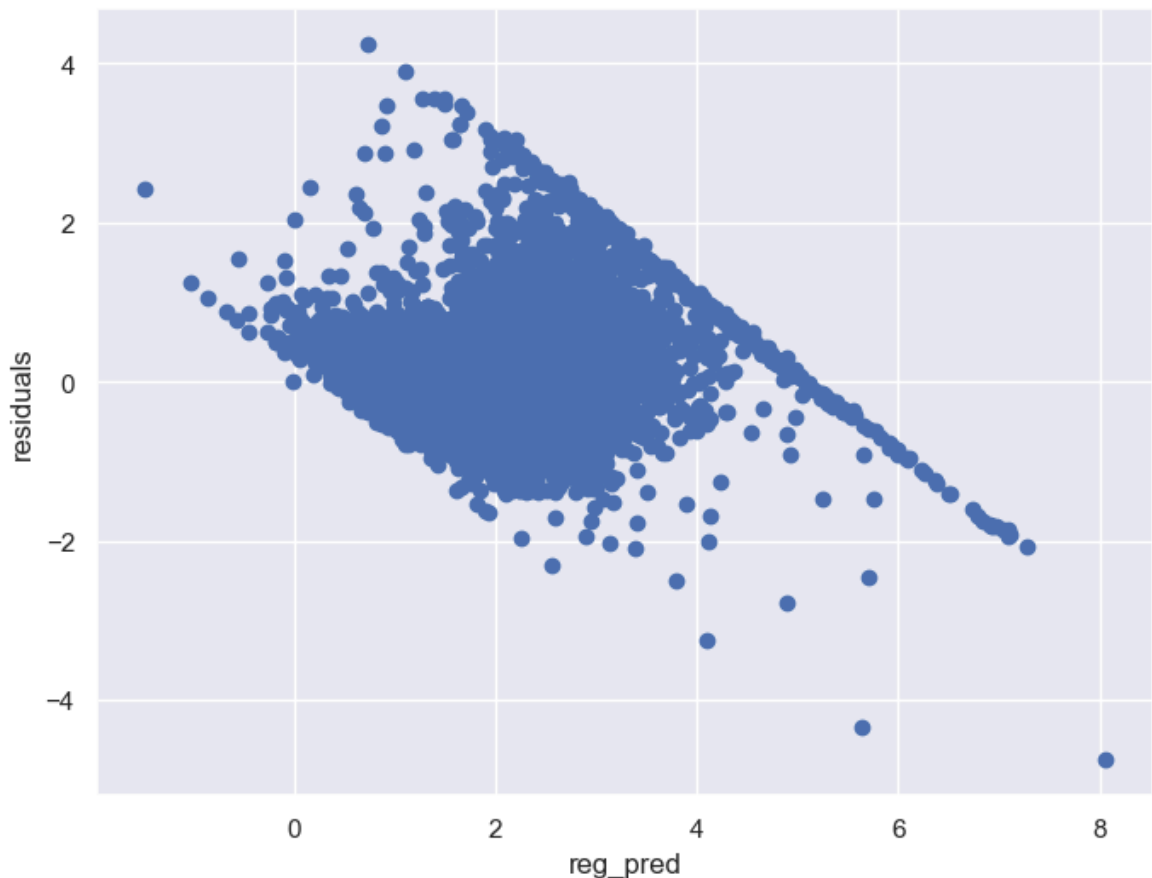


In [77]: `## residuals`
`## 2. Plot the distribution of residual errors - it should approximately follow`
`residuals = y_test - lasso_pred`
`sns.displot(residuals, kind="kde")`

Out[77]: `<seaborn.axisgrid.FacetGrid at 0x1c58aba0a30>`



```
In [78]: ## 3. Scattter plot with prediction and residual  
## uniform distribution  
plt.scatter(ridge_pred, residuals)  
plt.xlabel("reg_pred")  
plt.ylabel("residuals")  
plt.show()  
#No, this is not a uniform distribution.  
#It looks more like a concentrated triangular distribution along a sloping line  
# no shape randomly distribution called uniform distribution
```



4. ElasticNet

```
In [83]: from sklearn.linear_model import ElasticNet
elasticnet = ElasticNet(alpha=0.01)
```

```
In [84]: elasticnet.fit(X_train, y_train)
```

```
Out[84]: ElasticNet ⓘ ?
          Parameters
```

```
In [85]: elasticnet.intercept_
```

```
Out[85]: np.float64(2.0666362048018536)
```

```
In [86]: elasticnet.coef_
```

```
Out[86]: array([ 0.81596013,  0.127174 , -0.21749806,  0.260938 ,  0.          ,
                -0.03659241, -0.78750993, -0.75890644])
```

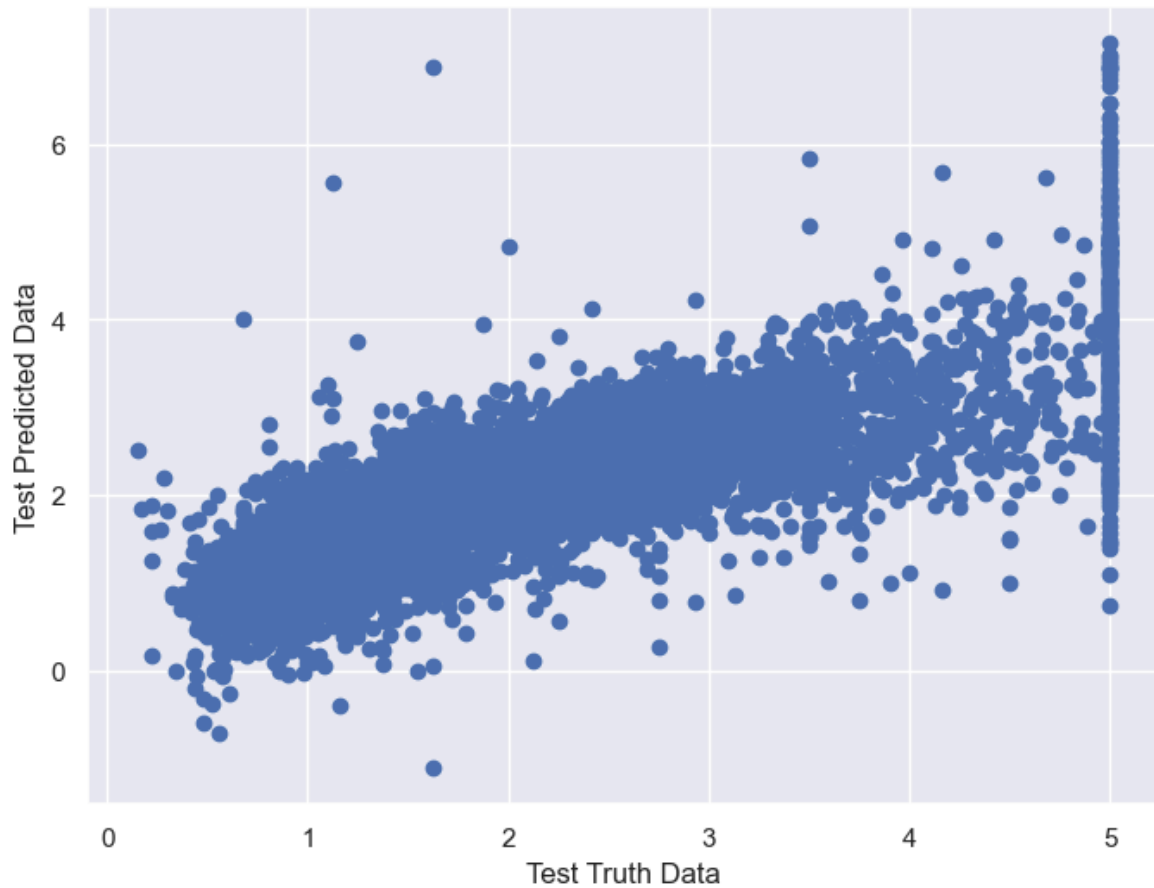
```
In [87]: elasticnet_pred = elasticnet.predict(X_test)
elasticnet_pred
```

```
Out[87]: array([0.76351523, 1.73146465, 2.63530683, ..., 1.34564033, 1.72109235,
                2.22289137], shape=(6812,))
```

```
In [88]: plt.scatter(y_test, elasticnet_pred)
plt.xlabel("Test Truth Data")
```

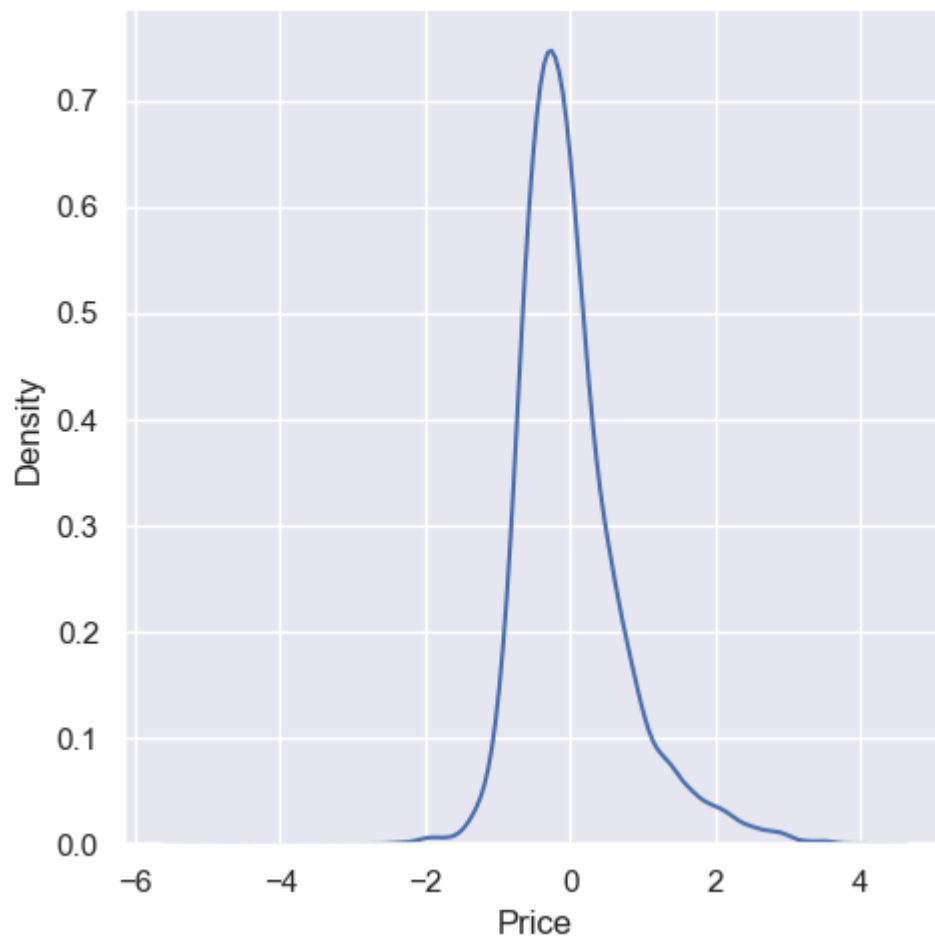
```
plt.ylabel("Test Predicted Data")
```

Out[88]: Text(0, 0.5, 'Test Predicted Data')

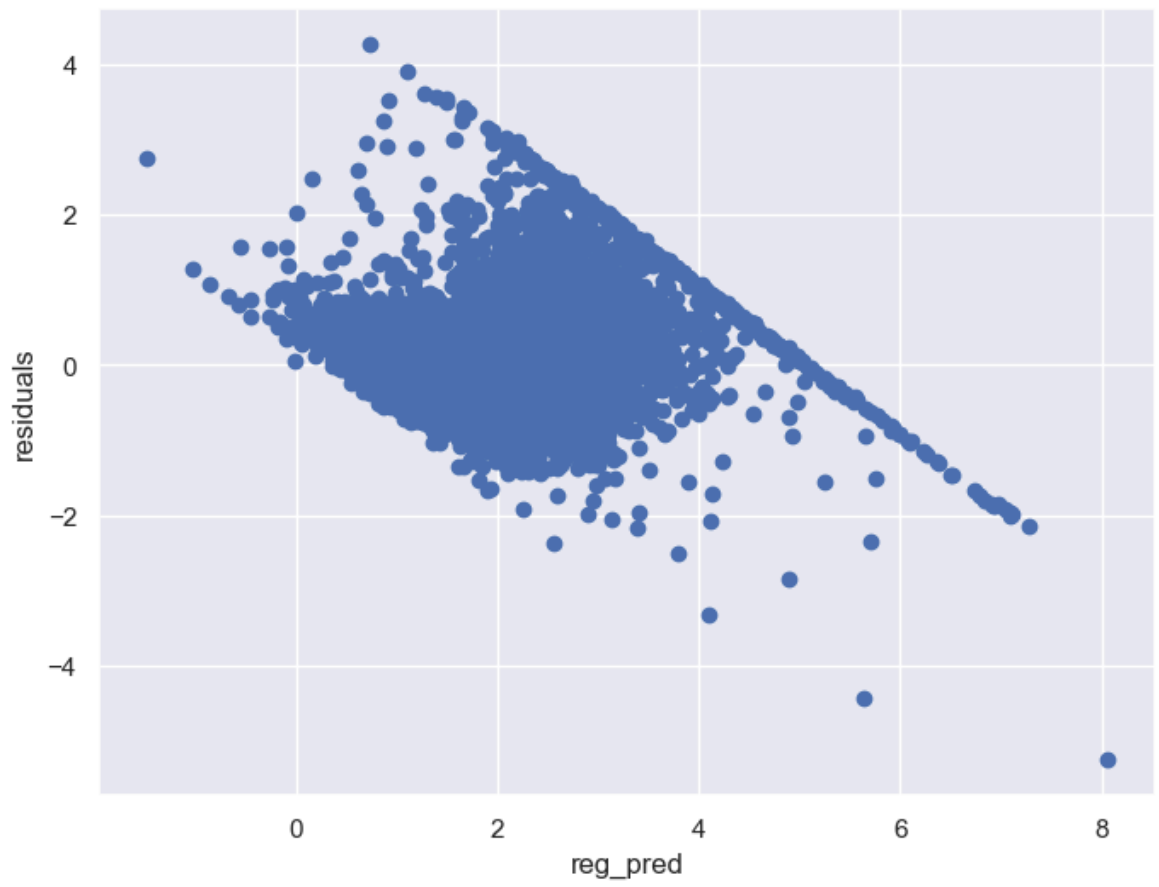


```
In [89]: ## residuals
## 2. Plot the distribution of residual errors - it should approximately follow
residuals = y_test - elasticnet_pred
sns.displot(residuals, kind="kde")
```

Out[89]: <seaborn.axisgrid.FacetGrid at 0x1c58ab7b2e0>



```
In [90]: ## 3. Scattter plot with prediction and residual  
## uniform distribution  
plt.scatter(ridge_pred, residuals)  
plt.xlabel("reg_pred")  
plt.ylabel("residuals")  
plt.show()  
#No, this is not a uniform distribution.  
#It looks more like a concentrated triangular distribution along a sloping line  
# no shape randomly distribution called uniform distribution
```



In []: